

RAG Chatbot

Installing and using the app

Marko Mirković 72m/24, Obrada prirodnog jezika

What it does

Answers questions about the documents in `data/raw/` and cites the passages it used.

1. Retrieves passages with one of four modes

- **BM25**: keyword search over stemmed words
- **Vector**: embedding similarity (`nomic-embed-text`)
- **Hybrid**: Reciprocal Rank Fusion of BM25 and vector
- **Rerank**: hybrid's top 20 re-scored by a cross-encoder, run with Hugging Face `transformers`

2. A local Ollama model (`llama3.2:3b`) writes the answer from those passages

Requirements

- Docker with Docker Compose
- 8 GB of memory available to Docker
- About 11 GB of free disk for images and models

Everything runs locally. No API keys needed.

Installation

```
git clone https://github.com/MarkoKillin/NLP-RAG-Project.git
cd NLP-RAG-Project
docker compose up --build
```

The first start:

- pulls `llama3.2:3b-instruct-q8_0` and `nomic-embed-text` through Ollama
- downloads the reranker `cross-encoder/ms-marco-MiniLM-L-6-v2`
- builds the index from `data/raw/`

About 4 GB of downloads, reused on later starts. The app opens once the log shows

`Starting Streamlit app.`

To show download progress: `docker compose exec ollama ollama pull llama3.2:3b-instruct-q8_0`

Using the app

Open <http://localhost:8501>

1. Pick a retrieval mode and `top_k` in the sidebar
2. Ask a question about the sample documents, for example:
"What value is the BM25 parameter b usually set to?"
3. Open **View sources** to see the retrieved passages, marked `[cited]` where the answer used them

On macOS, Docker has no GPU access, so an answer takes about 20 seconds.

Configuration

Retrieval mode

bm25



Chunks retrieved (top_k)

5

BM25. Keyword search over stemmed words.

Vector. Embedding similarity.

Hybrid. Reciprocal Rank Fusion of both rankings.

Rerank. Hybrid's top 20, re-scored by a cross-encoder.

RAG Chatbot

Ask a question about the indexed documents



Sample data

`data/raw/` comes with 12 documents to test with:

- 9 text and Markdown files: information retrieval, word embeddings, transformers, evaluation metrics, NLP, coffee brewing, Git, HTTP status codes, the solar system
- 3 tables: `countries.csv`, `programming_languages.csv`, and `chemical_elements.xlsx` with two sheets

The 32 evaluation questions are written against these files.

Adding documents and changing settings

Put `.txt`, `.md`, `.csv` or `.xlsx` files in `data/raw/`, then:

```
docker compose restart rag-app
```

The index is rebuilt on every start. Each table row becomes one passage.

To change chunk size, `top_k`, models and more:

```
cp .env.example .env      # edit values  
docker compose up -d
```

Evaluation

With the app running:

```
docker compose exec rag-app python -m scripts.evaluate --k 3
docker compose exec rag-app python -m scripts.evaluate --ablation
docker compose exec rag-app python -m scripts.evaluate --grounding
```

- Recall@k, MRR@k and Hit@k for each retrieval mode
- `--ablation` : BM25 with stemming and stopwords removal on and off
- `--grounding` : checks whether each answer cites a passage that holds the answer